Clase 2. TESTING Y ESCALABILIDAD BACKEND

Optimización

CODERHOUSE

Objetivos de la clase

- Entender las implicaciones de un servidor en producción
- Teorizar la optimización general de un servidor
- Desarrollar un middleware de manejo de errores personalizados

CODERHOUSE

Rendimiento en producción

CODERHOUSE

La realidad de un servidor

Cuando comenzamos a “acostumbrarnos” a desarrollar módulos y funcionalidades en nuestro servidor, seguramente comenzamos a agregar múltiples cosas mientras continuamos aprendiendo a utilizar el universo de herramientas para nuestro servidor.

Sin embargo, cuando queremos mostrar nuestro servidor al mundo, nos encontramos con una terrible realidad: la experiencia del usuario puede verse afectada debido a la saturación de cálculos, herramientas y funcionalidades por cargar.

CODERHOUSE

Optimización del servidor

Para que el cliente pueda contar con una experiencia de usuario satisfactoria al utilizar nuestro sistema, podemos implementar algunas prácticas de optimización y hacer un flujo sólido y constante.

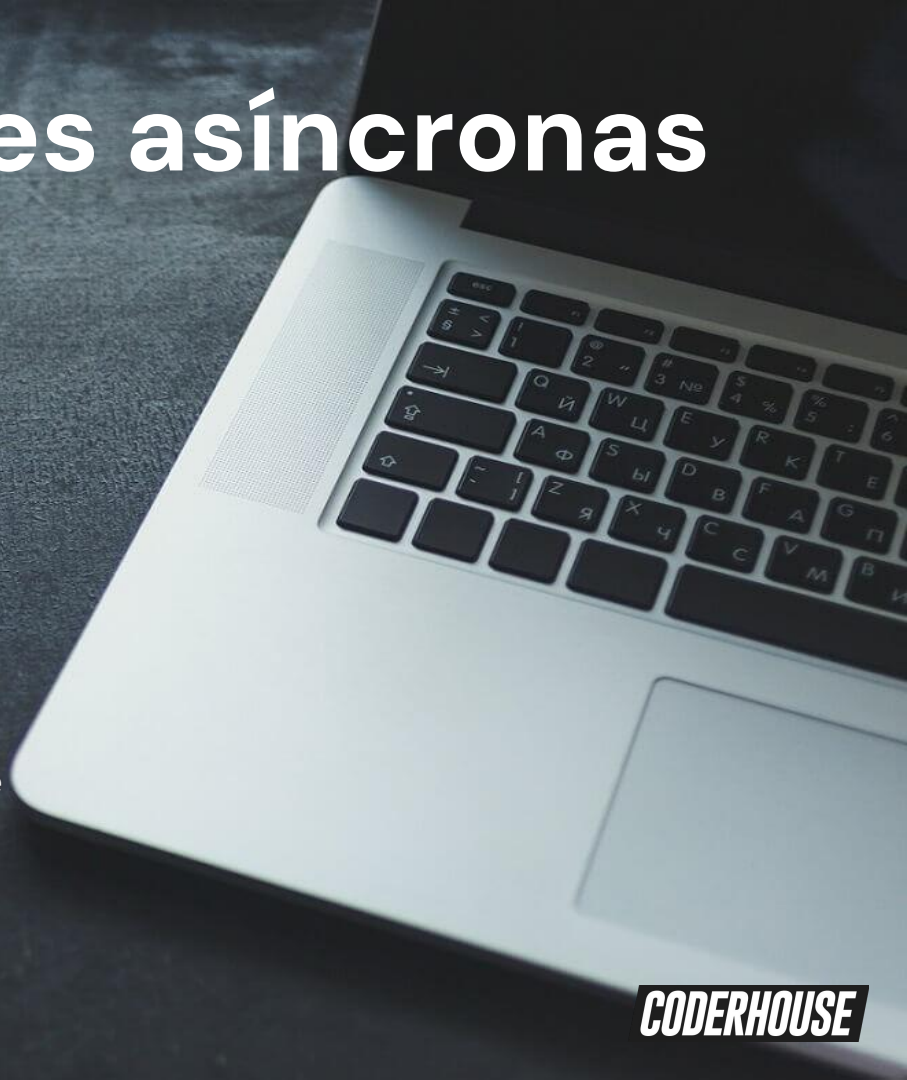
A continuación, se te brindarán un conjunto de prácticas que te ayudarán a asegurar un desarrollo de un servidor correctamente optimizado, sustentando una experiencia de usuario agradable.

CODERHOUSE

1. Utilizar funciones asíncronas

Como ya imaginarás, una de las razones por las que hemos utilizado funciones asíncronas todo este tiempo, es debido al carácter no bloqueante con el que éste cuenta en cuanto a sus operaciones.

Las funciones asíncronas permitirán que algunas otras tareas puedan ejecutarse mientras desarrollamos el trabajo principal, permitiendo hacer tareas como escribir archivos, leer datos de una base.



CODERHOUSE

2. Realiza un correcto loggeo

Aunque no lo pareciera, los console.logs() que vamos dejando a lo largo de nuestro flujo de desarrollo, pueden llegar a ser perjudiciales al hacer la medición del rendimiento de las tareas de nuestro servidor.

Un logger permitirá mostrar sólo lo necesario según el entorno en el que se corra el aplicativo, permitiéndonos loguear en consola cuando estemos desarrollando, pero ocultarlo cuando estemos en producción. Más adelante aprenderás cómo implementar un logger para tu servidor.

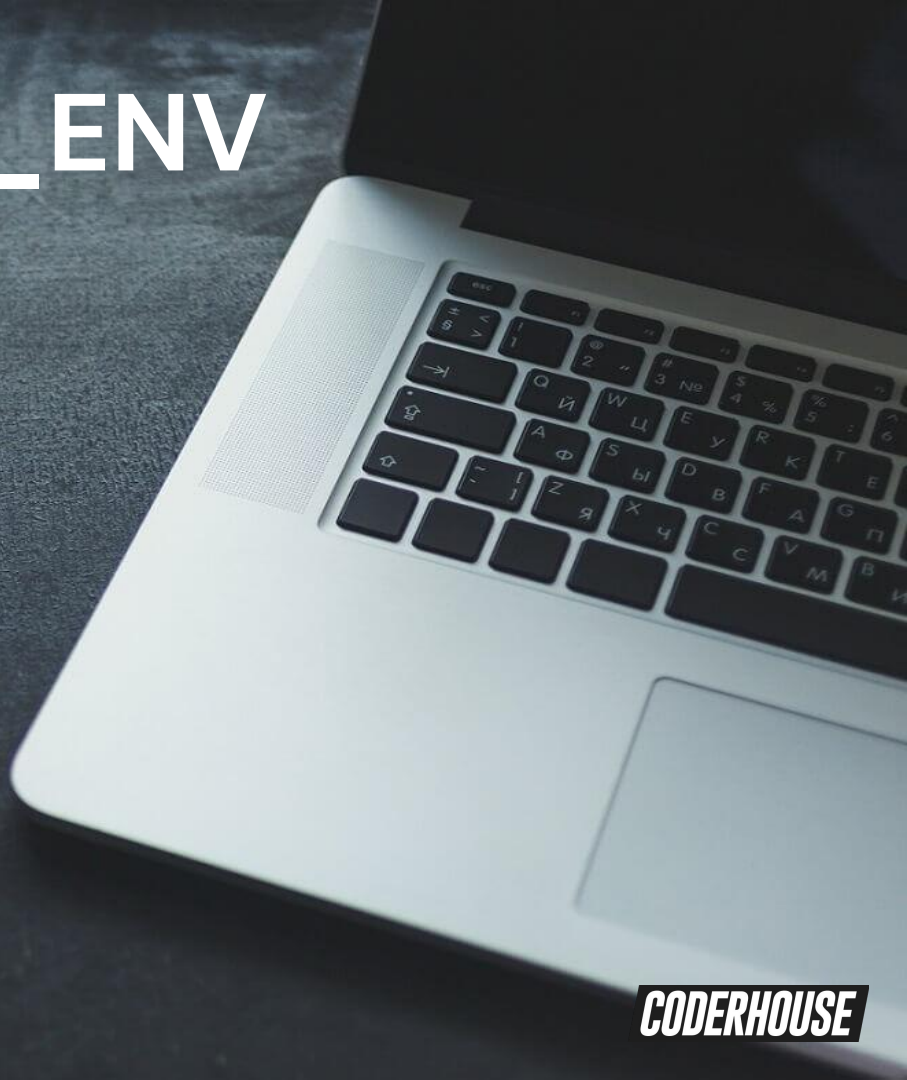
CODERHOUSE

3. Variable NODE_ENV

Express no es solo lo que configuramos, sino que también cuenta con una configuración interna que podemos aprovechar colocando esta simple variable en nuestro entorno.

Setear NODE_ENV = production hará que Express de manera interna cambie algunas cosas en su configuración, como es:

- Guardar en caché templates de vistas
- Guardar en caché también archivos de estilos de extensiones CSS



CODERHOUSE

4. Clusterizar el servidor

Existe una forma de aprovechar los hilos o CPUs de un computador, para levantar múltiples instancias del servidor y poder atender un mayor número de peticiones concurrentes.

La clusterización nos ayudará a tener un “equipo de aplicaciones” para atender las tareas más complicadas.

CODERHOUSE

5. Reinicio automático

En caso de que un error llegara a ocurrir a lo largo de la aplicación, nuestro servidor podría verse afectado al punto de que éste se caiga.

Si implementamos lo anterior, podremos mantener múltiples instancias de nuestra aplicación, aunque también podemos utilizar un manejador de procesos.

Un manejador de procesos nos permitirá escuchar por estas instancias de aplicación y reiniciar alguna de éstas en caso de que algo malo llegara a suceder con alguna.

CODERHOUSE

6. Manejo de errores

Recordemos que un error puede suponer algo fatal para el flujo de una aplicación. Ciertos errores pueden hacer inútil el continuar con los procesos y pueden dejar el servidor abajo mientras alguien lo levanta nuevamente.

Idealmente esperamos que nunca se caiga el servidor por ello necesitamos asegurar que nuestro código esté lo mejor controlado posible utilizando try/catchs, con promesas, e implementando algunos “error handlers” para impedir que lleguen a puntos críticos del servidor.

CODERHOUSE

7. Realizar balanceos de carga

Algunos servidores o algunas instancias pueden procesar más recursos que otros, por ello, cuando múltiples servidores trabajan en conjunto, debemos poder contar con la posibilidad de decidir a qué servidor le debería corresponder un mayor número de peticiones.

De esta manera, el servidor A puede soportar tres veces más que el servidor B y C, o bien podemos hacer que el servidor A se encargue de soportar las peticiones que correspondan a procesos más complejos, mientras que B y C pueden encargarse de las peticiones más simples.

CODERHOUSE

8. Realizar compresión

La compresión es un recurso utilizado para poder realizar un número más reducido de transferencia, aún manteniendo el mismo contenido de data.

La compresión permitirá que nuestros archivos puedan viajar a través de la red de manera más rápida, manteniendo la consistencia.

CODERHOUSE

9. Utilizar un proxy inverso

Utilizar un proxy inverso o un punto intermedio entre el servidor y el cliente, nos permitirá tener una mejor gestión de los servidores con los cuales estamos trabajando para el desarrollo de nuestra aplicación.

Un proxy inverso nos será útil para muchas cosas:

- Anonimato de servidores, generando un punto de identificación general y redireccionando a los servidores.
- Realizar carga de archivos estáticos, para reducir la carga de trabajo al servidor
- Realizar balances de carga y compresiones.

CODERHOUSE

¡Importante!

¡Aún hay mucho por aprender! Una vez que nuestro servidor ha cumplido con su función principal, es momento de comenzar a pensar en optimizarlo, mantenerlo y prepararlo para entornos empresariales reales.

CODERHOUSE

Compresión

CODERHOUSE

Compresión

Cuando el servidor se está comunicando con el cliente en el navegador, parte de esta comunicación implica revisar si hay algún archivo comprimido que necesite descomprimirse, y en caso de que así sea, cuál sería el algoritmo de descompresión a utilizar para poder obtener la información correctamente.

Los navegadores modernos aceptan contenido codificado en tres algoritmos principales:

- Deflate
- **Gzip (Deflate + algunas cosas adicionales)**
- **Brotli**

CODERHOUSE

Compresión

Gzip es el primer y más conocido modelo de compresión, es altamente utilizado y sencillo de utilizar.

La compresión Gzip es especialmente eficaz en tipos de datos que contienen repeticiones o patrones, como texto y datos HTML. Los archivos CSS, JavaScript y JSON también suelen comprimirse bien con Gzip.

CODERHOUSE

Compresión

Al reducir el tamaño de los archivos antes de la transferencia, Gzip ayuda a acelerar la carga de páginas web y reduce el consumo de ancho de banda, lo que es beneficioso tanto para los proveedores de contenido como para los usuarios finales.

La mayoría de los navegadores web modernos admiten la decodificación Gzip de manera automática, por lo que no es necesario que los usuarios realicen ninguna acción adicional.

CODERHOUSE

Compresión

Para poder utilizarlo, instalaremos el módulo compression:
npm i express-compression

Gzip se implementa fácilmente como un middleware de terceros:

```
import express from "express";
import compression from "express-compression";

const server = express();

server.use(compression());

export default server;
```

CODERHOUSE

Compresión

Name

stringridiculamentelarga

× Headers Preview Response Initiator Timing

General

Request URL: http://localhost:8080/stringridiculamentelarga
Request Method: GET
Status Code: 200 OK
Remote Address: [::1]:8080
Referrer Policy: strict-origin-when-cross-origin

Response Headers View source

Connection: keep-alive
Content-Encoding: gzip
Content-Type: text/html; charset=utf-8
Date: Wed, 26 Oct 2022 04:42:31 GMT
ETag: W/"256280-fD1Y30RmdTf1MruftVdoPuIJ0OI"
Keep-Alive: timeout=5
Transfer-Encoding: chunked
Vary: Accept-Encoding
X-Powered-By: Express

Request Headers View source

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
Accept-Encoding: gzip, deflate, br

Indica que se aceptó el gzip configurado

CODERHOUSE

Compresión

Brotli es conocido como una alternativa “moderna” de Gzip. Fue desarrollado por Google y ofrece un algoritmo cuya compresión puede resultar hasta 30% más efectiva que la compresión de Gzip

Para ello, podemos seguir utilizando la dependencia de express-compression que recién instalamos, añadiendo un poco de configuración extra:

```
server.use(compression({
      brotli: { enabled:true, zlib:{} }
}));
```

CODERHOUSE

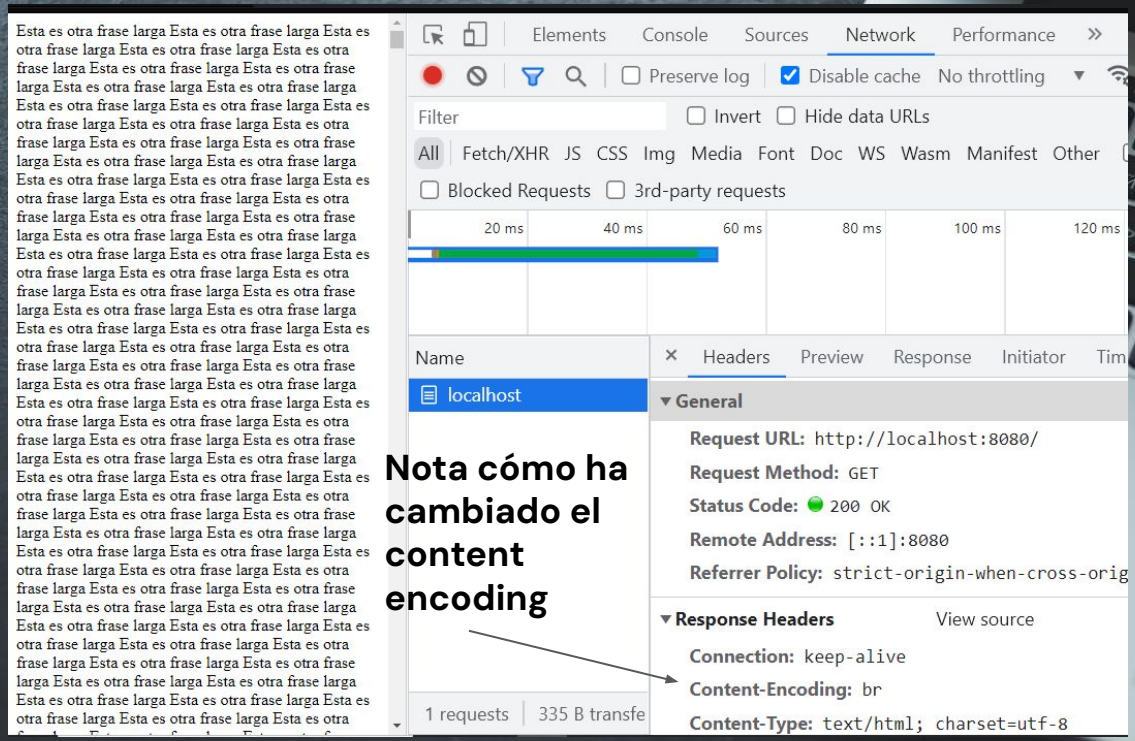
Compresión

Con esta opción, se reconocerá un tipo de compresión “br” (brotli) al momento de enviar la información.

La razón por la que colocamos un objeto zlib vacío se debe a que el módulo de **express-compression** cuenta con una dependencia interna “**zlib**”, la cual le permite ejecutar diferentes niveles de compresión.

CODERHOUSE

Compresión



Nota cómo ha cambiado el content encoding

CODERHOUSE

¡Importante!

La compresión de brotli es considerablemente más efectiva que gzip. **Sin embargo,** utilizarlo como middleware implica un uso del algoritmo de compresión que consume más recursos, por lo que existe una **nivelación entre tiempo de compresión y efectividad.**

CODERHOUSE

Middleware para manejo de errores

CODERHOUSE

Errores

Aún con amplios años de experiencia, la probabilidad de cometer un error a lo largo de tu carrera como desarrollador nunca será cero.

Es por ello que, parte de tener un servidor correctamente optimizado, es también tener un correcto control de los errores que pueden llegar a ocurrir en el sistema.

La idea será contar con nuestra propia gestión interna de errores, con el fin de mantener un mejor orden en el equipo de desarrollo.

CODERHOUSE

Errores

Contar con una librería de errores permitirá que todos los desarrolladores puedan tener una referencia de los errores más comunes dentro de su aplicativo, apoyando así a su respectiva depuración

Para poder generar este flujo de error, necesitaremos de tres cosas primordiales:

- Un diccionario de errores.
- Un generador personalizado de errores.
- Un middleware de recepción de errores.

CODERHOUSE

Diccionario de errores

```
const errors = {
 error: { message: "Error", statusCode: 400 },
 auth: { message: "Bad auth", statusCode: 401 },
 forbidden: { message: "Forbidden", statusCode: 403 },
 notFound: { message: "Not found", statusCode: 404 },
 fatal: { message: "Fatal", statusCode: 500 },
};

export default errors;
```

CODERHOUSE

Error Personalizable

```
class CustomError {
 static new({ message, statusCode }) {
   const error = new Error(message);
   error.statusCode = statusCode;
   throw error;
 }
}

export default CustomError;
```

CODERHOUSE

Manejador de errores

```
import errors from "./errors/errors.js";

const errorHandler = (error, req, res, next) => {
 console.error(error);
 const statusCode = error.statusCode || errors.fatal.statusCode;
 const message = error.message || errors.fatal.message;
 return res.status(statusCode).json({
   message,
   url: `${req.method} ${req.url}`,
 });
};

export default errorHandler;
```

CODERHOUSE

Break

CODERHOUSE

ACTIVIDAD EN CLASE

Errores frecuentes

Definir:

- ✓ diccionario de errores frecuentes
- ✓ error personalizado
- ✓ middleware de manejo de errores
- ✓ middleware de autorización

Implementar los errores frecuentes de “no encontrado”, “autenticación” o “autorización” según corresponda.
Implementar el middleware de autorización donde corresponda.

CODERHOUSE

¡Pronto llega Docker!

Instalar Docker

¿Qué estoy por instalar?

Instalar curl

¿Qué estoy por instalar?

Ver tutorial

Página oficial

CODERHOUSE

Resumen
de la clase hoy

- ✓ Qué es el TDD
- ✓ Qué son los mocks
- ✓ faker js
- ✓ Aplicar los conceptos en una mocking API sencilla.

CODERHOUSE

#DemocratizandoLaEducación

CODERHOUSE